

# Programación Avanzada

## TEO 1

Presentación API Rest Minimarket

---

**Branco Merino Huerta**

**Carrera: Ingeniería en Informática.**  
**Profesor: Agustín Díaz.**

# Introducción

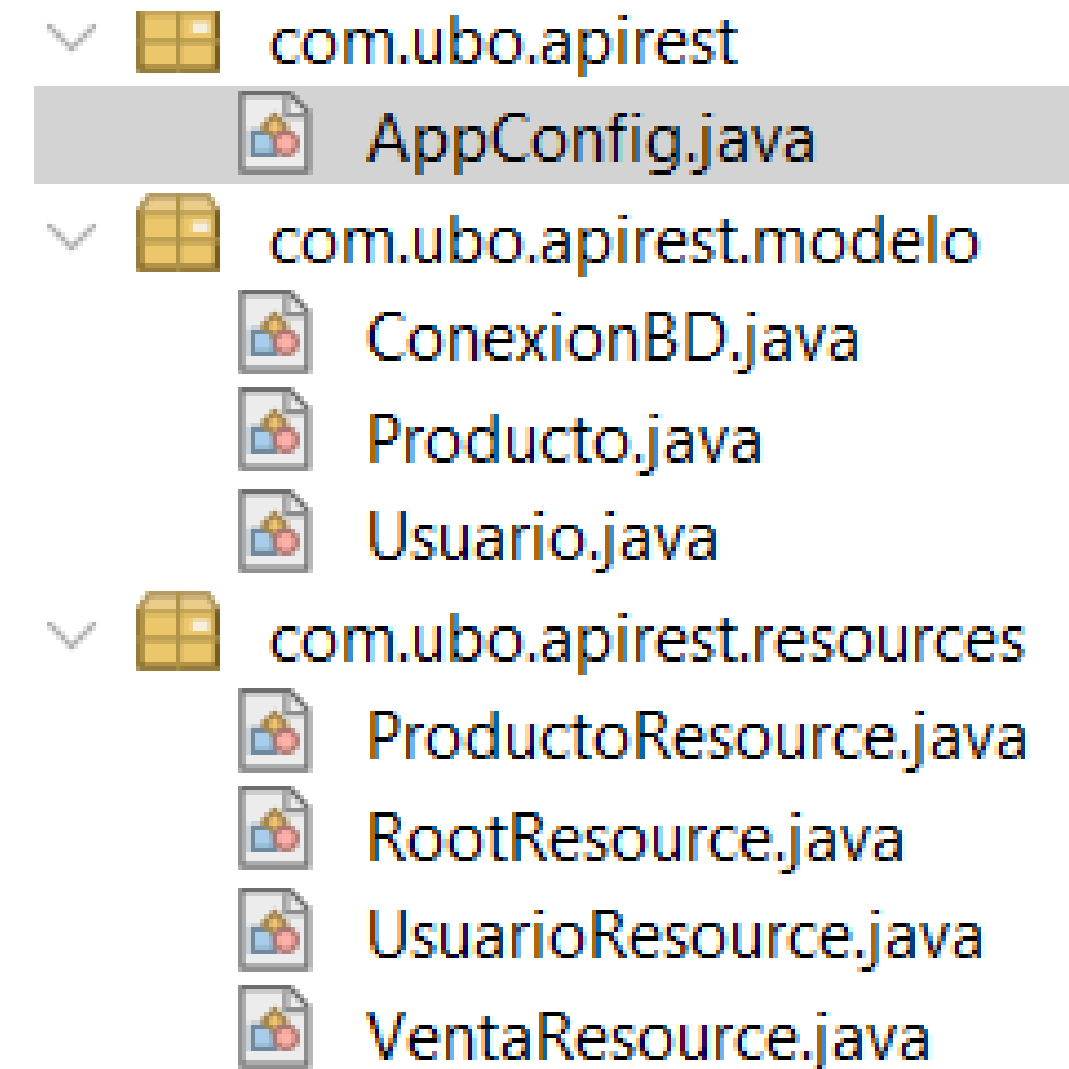
Este proyecto tiene como objetivo desarrollar e implementar una API REST funcional que permita gestionar los productos y las ventas para un minimarket. Esta API servirá como capa intermedia entre la base de datos y las aplicaciones cliente (como Postman), proporcionando un acceso seguro y organizado a los datos.

El proyecto utiliza Java como lenguaje principal y JSON como formato de intercambio de datos, junto con una base de datos relacional MySQL conectada mediante JDBC.

# Estructura

- Configuración: AppConfig detecta las clases con @Path y activa los servicios REST.
- Modelo: Entidades del sistema y la conexión a la base de datos (JDBC).
- Resources (API REST): Endpoints organizados por entidad (productos, usuarios, ventas).
- Formato JSON: Entrada y salida de datos.

**Ruta: <http://localhost:8080/ApiMinimarket>**



# Descripción Endpoints

## Endpoints de Productos (ProductoResource)

### Crear nuevo producto

**Método: POST**

**Ruta: /api/productos**

JSON

```
{  
  "nombre": String,  
  "idCategoria": Int,  
  "tipoVenta": String,  
  "precioBase": Double,  
  "stock": Int,  
  "extra": String  
}
```

### Actualizar producto

**Método: PUT**

**Ruta: /api/productos/{id}**

JSON

```
{  
  "nombre": String,  
  "idCategoria": Int,  
  "tipoVenta": String,  
  "precioBase": Double,  
  "stock": Int,  
  "extra": String  
}
```

### Eliminar producto

**Método: DELETE**

**Ruta: /api/productos/{id}**

# Descripción Endpoints

## Endpoints de Usuarios(UsuarioResource)

### Login de Usuario

Método: POST

Ruta: /api/usuarios/login

```
JSON
{
  "username": String,
  "password": String
}
```

## Endpoints de Ventas(VentaResource)

### Registrar Venta

Método: POST

Ruta: /api/ventas

```
JSON
{ "subtotal": Double,
  "iva": Double,
  "total": Double,
  "idUserio": Int,
  "detalles": [ {
    "idProducto": Int,
    "cantidad": Int,
    "precioUnitario": Double
  } ] }
```

### Listar Ventas

Método: GET

Ruta: /api/ventas

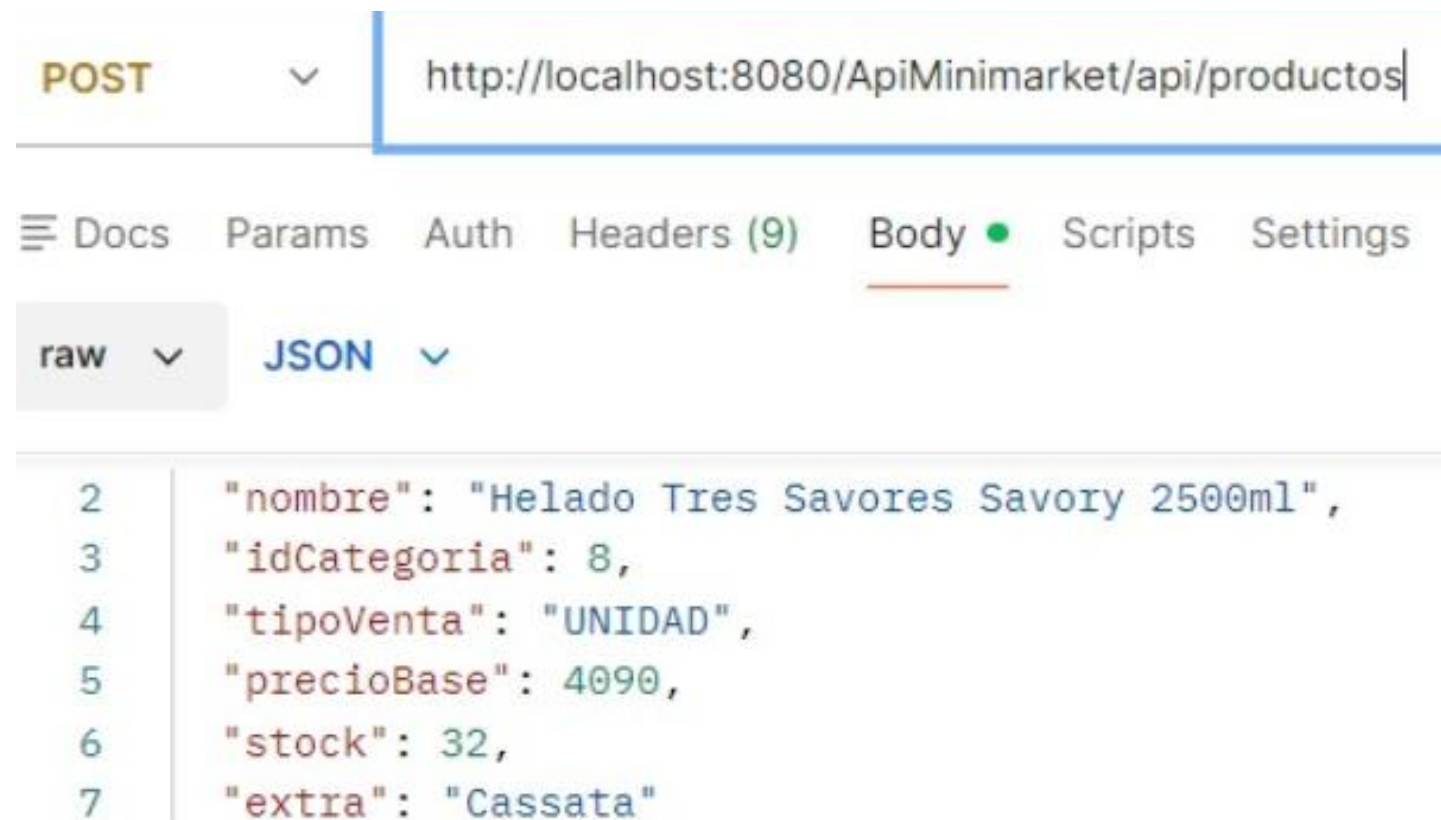


# Flujo de datos

## Agregar Producto

Se envia un JSON desde  
Postman

El metodo crearProducto recibe el JSON y realiza la  
consulta sql



```
public Response crearProducto(Producto producto) {

    String consultaSQL = "INSERT INTO productos " +
        "(nombre, id_categoria, tipo_venta, precio_base, stock, extra) " +
        "VALUES (?, ?, ?, ?, ?, ?)";

    try {
        Connection con = ConexionBD.conectar();
        PreparedStatement ps = con.prepareStatement(consultaSQL);
```

## Agregar Producto

Si la consulta se realiza  
correctamente:

```
JSONObject result = new JSONObject();
result.put("mensaje", "Producto creado correctamente");
result.put("idProducto", producto.getIdProducto());
result.put("nombre", producto.getNombre());
result.put("idCategoria", producto.getIdCategoria());
result.put("tipoVenta", producto.getTipoVenta());
result.put("precioBase", producto.getPrecioBase());
result.put("stock", producto.getStock());
result.put("extra", producto.getExtra());

return Response.status(Response.Status.CREATED).entity(result);
```

Si no se puede realizar el insert:

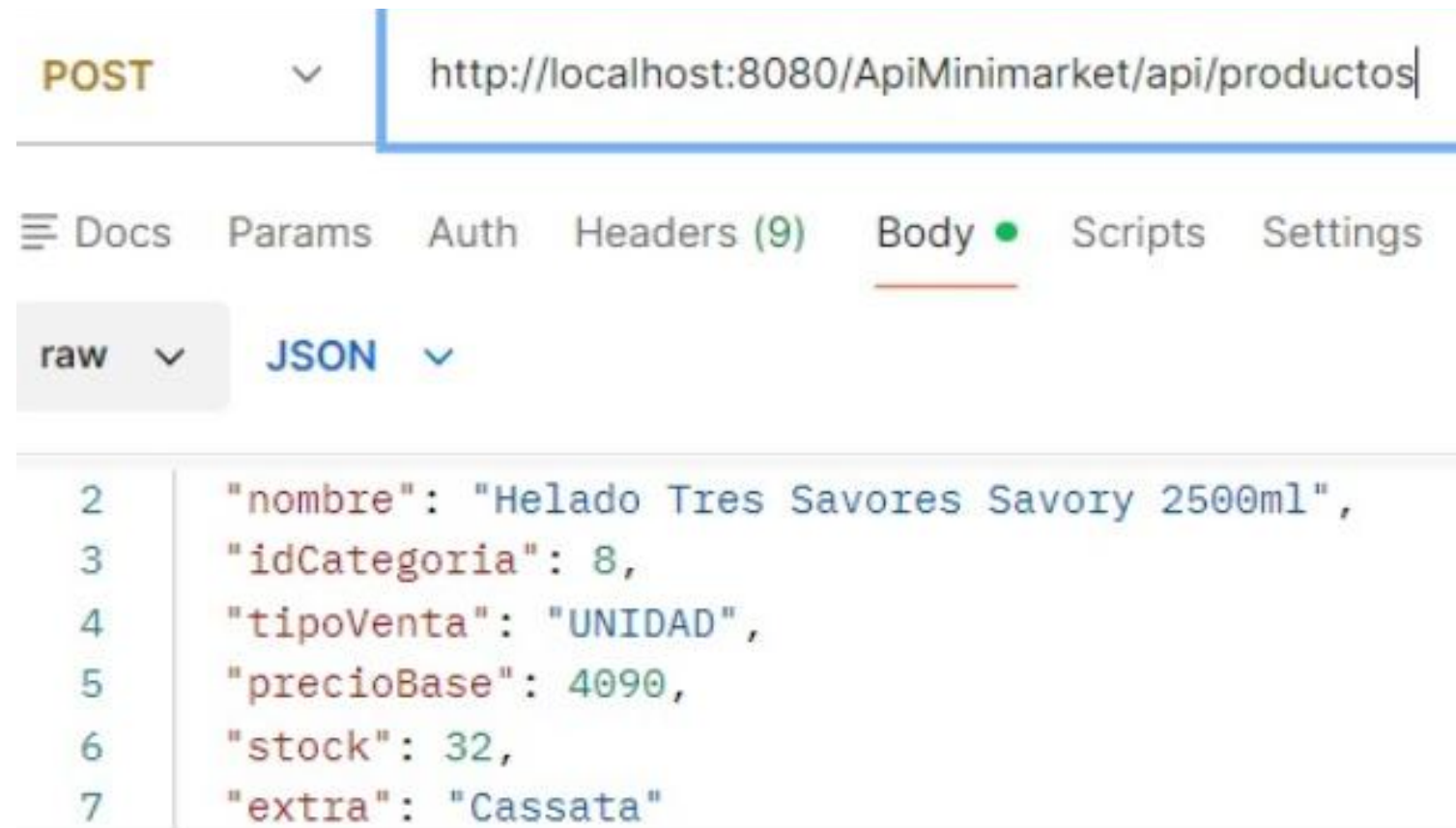
```
int filas = ps.executeUpdate();
if (filas == 0) {
    JSONObject error = new JSONObject();
    error.put("Error", "No se pudo insertar el producto");
    return Response.status(Response.Status.BAD_REQUEST).entity(error);
}

catch (SQLException e) {
    JSONObject error = new JSONObject();
    error.put("ERROR", "Error al crear producto" + e.getMessage());
    return Response.serverError().entity(error.toString()).build();
}
```



# Ejemplo agregar producto

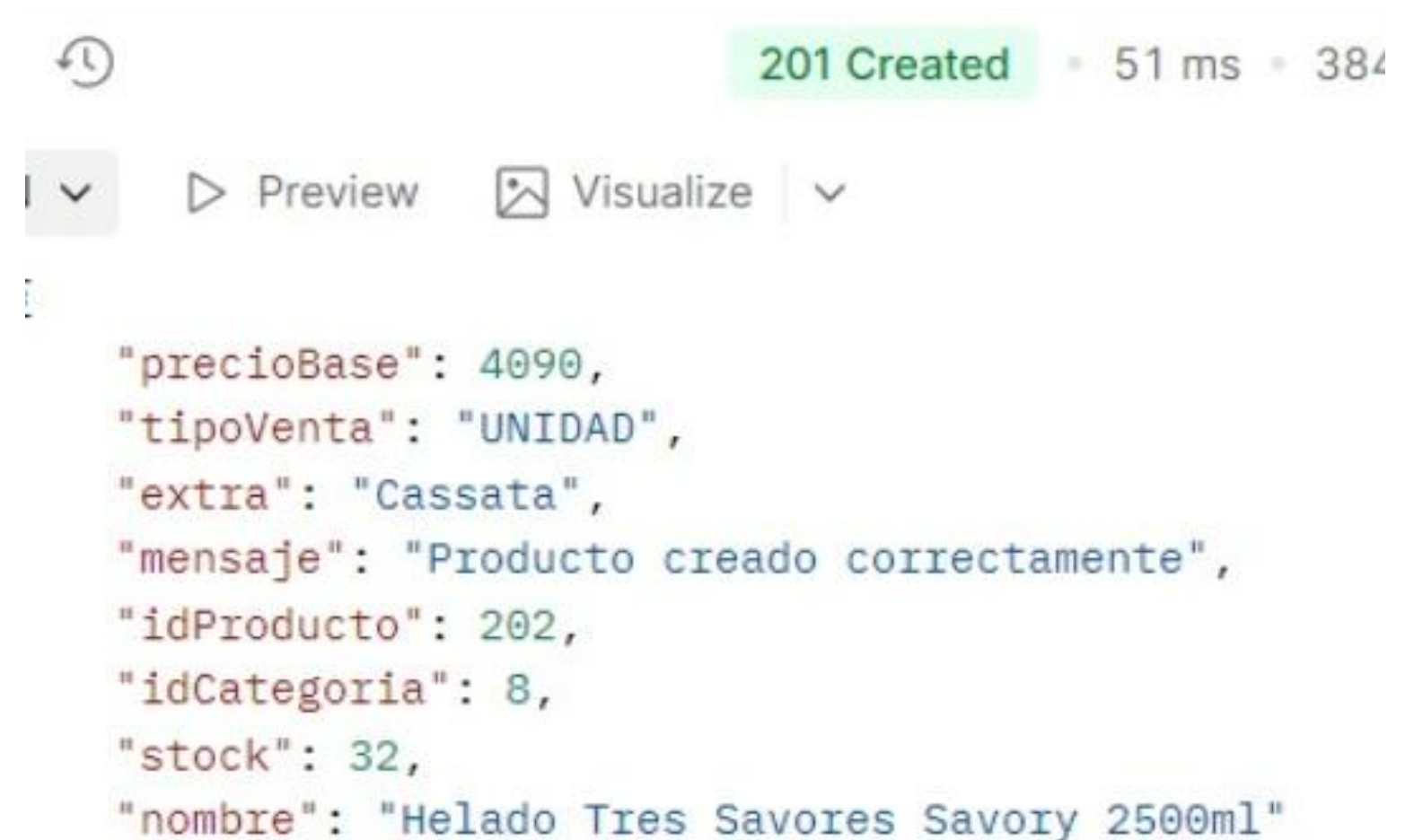
Se envían los datos del producto a través de un JSON



```
POST http://localhost:8080/ApiMinimarket/api/productos

{
  "nombre": "Helado Tres Saborés Savory 2500ml",
  "idCategoria": 8,
  "tipoVenta": "UNIDAD",
  "precioBase": 4090,
  "stock": 32,
  "extra": "Cassata"
}
```

Se recibe un JSON confirmando la creación del producto nuevo



```
201 Created - 51 ms - 384

{
  "mensaje": "Producto creado correctamente",
  "idProducto": 202,
  "idCategoria": 8,
  "stock": 32,
  "nombre": "Helado Tres Saborés Savory 2500ml"
}
```

El producto aparece en la base de datos correctamente

| id_producto | nombre                            | id_categoria | tipo_venta | precio_base | stock  | extra | activo | created_at          | updated_at          |
|-------------|-----------------------------------|--------------|------------|-------------|--------|-------|--------|---------------------|---------------------|
| 202         | Helado Tres Saborés Savory 2500ml | 8            | UNIDAD     | 4090.00     | 32.000 | NULL  | 1      | 2025-11-24 18:05:56 | 2025-11-24 18:05:56 |



# Ejemplo Error al agregar producto

**No se envían los datos necesarios del  
producto  
(Falta el nombre del producto)**

**Se recibe un JSON con el error**

```
{  
  "idCategoria": 8,  
  "tipoVenta": "UNIDAD",  
  "precioBase": 4090,  
  "stock": 32  
}
```

```
ON v Preview Debug with AI v  
{  
  "ERROR": "Error al crear producto Column 'nombre' cannot be null"  
}
```

# Conclusiones

**La implementación de la API REST permitió integrar correctamente la lógica del negocio con una aplicación para gestionar APIs, facilitando la comunicación entre el cliente y el servidor.**

**Se logró implementar correctamente las operaciones CRUD sobre la gestión de los productos utilizando Java, JSON y una conexión a una base de datos MySQL.**

# Posibles mejoras

Limitar el acceso a las APIs dependiendo del rol

Mejorar la actualizacion de productos para solo ingresar los datos a cambiar

Integrar un registro de las consultas realizadas (Insert, Update y Delete) y el usuario que las realizó

**Muchas gracias  
por su atención!!!**